Neuro-fuzzy Computing - Project

Giorgos Kakepakis - 03198, Giorgos Gantsios - 03283

February 2024

1 Introduction

Classification is one the the fundamental utilizations of machine learning models such as neural networks. In this project we are called to create a neural network model to perform hierarchical classification of 2 levels for texts, and specifically news documents. The dataset we use ('news-classification.csv') consists of 10917 text samples, which are classified in 17 main classes ('category_level_1') and 109 total sub-classes ('category_level_2'). This project was developed within the framework of the course of 'Neuro-Fuzzy Computing' (ECE-447) for University of Thessaly in February/March 2024.

2 Overview of Architecture/System

2.0.1 Approach

In order to resolve the task by utilizing the hierarchical relationships of the data we decided to use Local Classifiers, and specifically **Local Classifiers per Parent Node (LCPN)**. This is a hierarchical-structured system that consists of 2 levels of classifiers. Essentially we use one classifier for every class that is divided into sub-classes. The 1st level decides the main class of the input and based on this prediction "sends" it to the corresponding 2nd-level classifier who in turn decides the sub-class of the input.

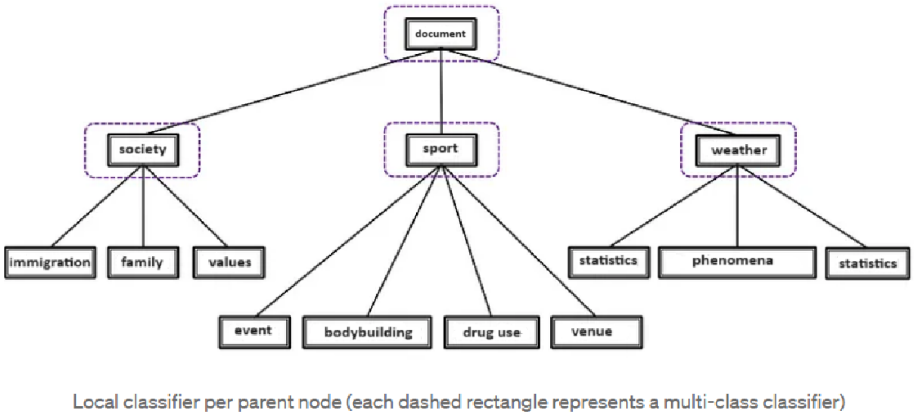

Figure 1: LCPN Classification

We use 1 classifier on the 1st level and 17 in the 2nd level, since there are 17 main classes. Every "internal node" represents a main class and every "leaf" represents a sub-class. **Each classifier is a separate neural network**.

2.0.2 Advantages

The LCPN approach has many advantages compared to other methods. Firstly, it is the most intuitive approach, and it takes advantage of the hierarchical nature of the data compared to a flat approach that would only use the 2nd level as target. In addition, there are no inconsitencies since the main class is predicted first, and the possible result is a sub-class that always belongs to this specific main class.

2.0.3 Disadvantages

The main drawback of this method is the **error propagation**. This is the case where the 1st-level prediction is wrong. If that happens the input is send to the wrong 2nd-level classifier, so the final result will definitely be wrong. This can be fixed during training but it remains a problem in testing. Also since we use multiple classifiers, hence neural networks, a lot of memory and time is required to store and train the models respectively.

3 Neural Networks Used

For both levels of the classification task we used RNNs, specifically **Bidirectional LSTMs**. Before choosing the model type we also worked with CNNs, and LSTMs performed better, especially on the 2nd-level where the training data is fewer. In addition, the addition of bidirectional LSTMs instead of regular LSTMs proved to improve the performance significally, since it captures the context better by processing the text in both ways.

3.0.1 1st Level

This level consists of only 1 classifier. The network consist of: the embedding layer (more on in on "Data Preprocessing"), 1 bidirectional LSTM layer and a dense output layer. The number of cells in the LSTM layer is **150**

3.0.2 2nd Level

As we mentioned we use 17 classifiers int the 2nd level, one for each main class to sub-class connection. Similarly to the 1st level, the network consists of: the embedding layer, 1 bidirectional LSTM layer and a dense output layer. The number of cells in the LSTM layer is now **100** for almost each network, and specifically:

Table 1: LSTM cells per classifier

Classifier index 1	LSTM Cells
1	100
2	100
3	100
4	100
5	100
6	100
7	50
8	150
9	100
10	100
11	100
12	100
13	100
14	150
15	100
16	100
17	100

4 Dataset Analysis

4.0.1 Data Balance/Imbalance

First of all we should examine the formation of our dataset. If we look at the 2nd level of classification we see that there is almost perfect balance, since nearly every separate sub-class comprises exactly 100 samples. This is beneficial for the 2nd-level classifiers because they do not have to deal with data imbalances. For example:

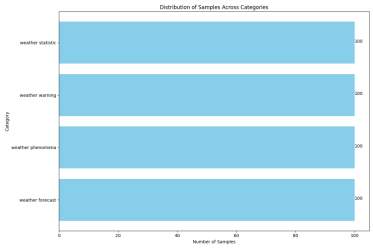

Figure 2: Weather Class

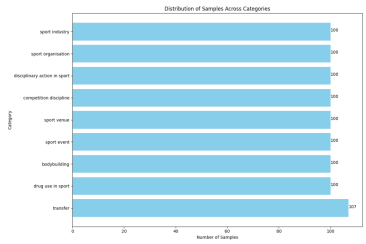

Figure 3: Sport Class

However, this creates data imbalances between main classes that consist of different amount of sub-classes. If we look at the 2 previous Figures, the total

samples of the class "Sport" will be 907, in comparison to the class "Weather" which only has 400.

Here is the total Data Distribution across the whole dataset:

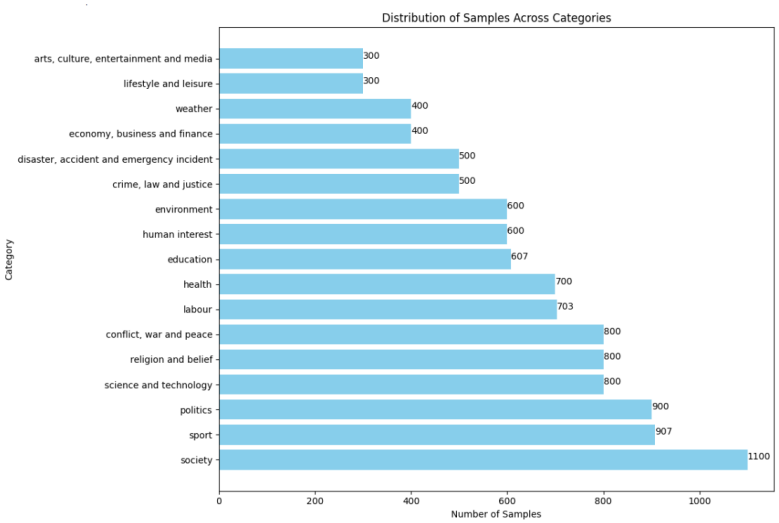

Figure 4: Data Distribution

The way we handled the data imbalance is the use of **class weights** during the training of the 1st-level classifier. The concept of class weights is to assign a higher weight to the classes with fewer instances and a lower weight to the classes with more instances. By doing so, the loss function used during training is adjusted to penalize misclassifications of the minority class more than misclassifications of the majority class. We use the **'Scikit-learn'** python library to implement this method in our code.
We also tried to perform **oversampling** on the minority classes in order to get rid of the data imbalance, but it led to overfitting and worse generalization.

4.0.2 Datasets for Training/Testing

Let's see how we use the original dataset to train the classifiers. From now on we shall refer to the 1st-level classifier as "main classifier", and the 2nd-level classifiers as "sub-classifiers" for convenience. First of all, we initially split the original dataset and kept **10%** for **testing**. While doing that we secured that we maintained the same ratio of every class sample in both the test and the training dataset.

For the 1st level, we just use the training dataset we mentioned along with the class weights to train the main classifier.

For the 2nd level, we split the training dataset to 17 'sub-datasets'. Each one is dedicated to a distinct main class and contains only the samples that belong to it. Since each sub-class has always about 90 samples, these 'sub-datasets' are always **balanced**. We use one such 'sub-dataset' for each sub-classifier. In that way, the sub-classifiers don't have to deal with data imbalances, and are trained only on valid data of their corresponding main class.

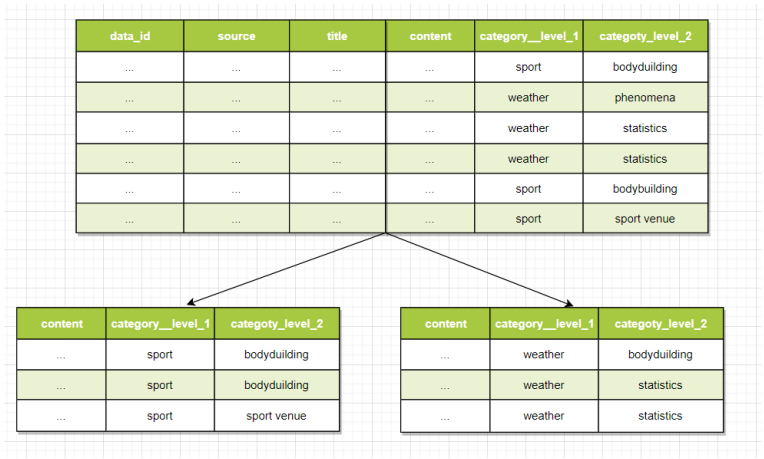

Figure 5: Dataset Split

Note: when we split the original dataset the sub-datasets keep all the columns, the previous illustration is simplified.

5 Data Preprocessing

Let's see how we preprocess the input texts in order to turn them into a form applicable to to the model:

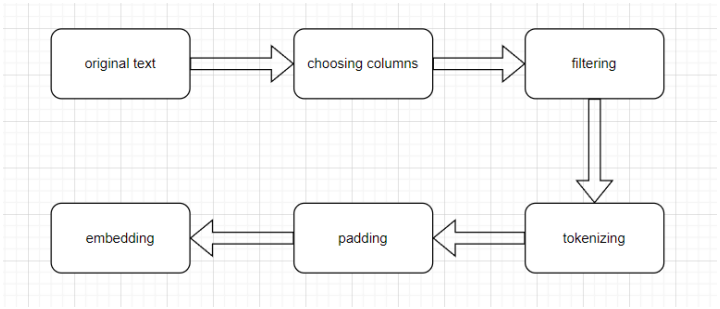

Figure 6: Preprocessing Workflow

5.0.1 Choosing Columns

First of all we must choose which data we are going to use from each entry. Obviously we have to use the text (**'content'**), but we also use the **'title'**, **'source'** and **'id'**, since we believe that they provide important information. When concatenating we put the 'content' last, since the other columns contain a lot of important context in just a few words, and we don't want to risk losing them during the padding.

5.0.2 Filtering

We use a function **'remove_stopwords'** which receives a text and removes words that do not contribute to the context, such as 'of', 'for', etc. This is very helpful since the output texts will contain words that will be useful, plus we reduce the size of the input saving memory.

5.0.3 Tokenizing

We create a new tokenizer which corresponds directly to this specific model and will be returned to be used in the testing phase. We apply it to the filtered text and it transforms it into a sequence of integers, where the value of each token depends on its frequency on the texts.

5.0.4 Padding

The input data of a neural network must have a fixed size. We choose a length that fully covers **90%** of the training tests, so we don't lose much information while simultaneously saving some memory. The percentage can be selected as a parameter while training the network.

5.0.5 Embedding

Before passing the data to the netowrk we want to use an embedding layer in order to capture relationships and contextual information. We use a **pre-trained embedding** layer, specifically **Word2Vec**. We allow the embedding

layer to be fine-tuned so they can capture domain-specific contexts and improve performance on tasks within that domain without starting from scratch.
The use of pre-trained embedding proved to be really usefull, especially for the 2nd level, where the training data is much more compromised.

6 Optimizer

The training algorithm we use is the **Adam optimizer**, which is widely used in training deep neural networks since it offers several advantages. It adaptively adjusts the learning rates for each parameter based on the magnitude of gradients like AdaGrad, while incorporating momentum optimization. We set the initial **learning rate** to **0.001**, but this can be modified though as a parameter in training.
We use **Categorical Crossentropy** as the loss function, since it is specifically designed for multi-class classification problems. It measures the dissimilarity between the predicted probabilities and the true probabilities, using the formula: $C(y, \hat{y}) = -\sum_i y_i \cdot \log(\hat{y}_i)$

7 Program Operation

We will now explain briefly how the program works and how it can be excuted. Firstly, there are many help functions created at the top of the file (**'Help Functions'**) used for every aspect of the program (training, evaluating, visualizing, etc). The main program functionality begins at the label 'Main program'.

In relation to the functinon that creates a new trained model ("build_trained_lstm_model"): the user is able to choose the parameters of the network, such as the number of LSTM layers, the cells used, the initial learning rate and more.

7.0.1 Main program - Initialization

This is where we start. These cells should be executed always **at the start of the program** because they initialize objects that the program uses constantly, such as the mapping of the main classes and sub-classes (**'Encoding of Main-Classes/Sub-Classes'**).
This is where the user should choose the input (location of the datasets) and output directory of the program (**'Set the base-filepaths'**).

7.0.2 Main program - Main Classifier

This is where we process the main classifier. There are the following options:

- **'Train the Main Classifier'**: the user can create a new trained LSTM choosing the desired parameters

- **'Save the Main Classifier'**: the user can save the new trained LSTM in the output directory
- **'Load the Main Classifier'**: the user can load a pre-trained LSTM stored on a file, in the input directory
- **'Evaluate the Main Classifier'**: the user can apply the test dataset to check the performance of the classifier

7.0.3 Main program - Sub-Classifiers

This is where we process the sub-classifier. There are the following options:

- **'Train the Sub-Classifiers (at once)'**: the user can create new trained LSTMs choosing the desired parameters, the number of units for each LSTM can be specified in the list "num_lstm_units_list". This cell returns the list "trained_models_info", which contains the information needed to evaluate the trained models (tokenizer, input length)
- **'Save the Sub-Classifiers (at once)'**: the user can save the new trained LSTMs in the output directory
- **'Load the Sub-Classifiers (at once)'**: the user can load pre-trained LSTMs stored on files, in the input directory
- **'Train the Sub-Classifiers (separately)'**: the user can create just one new trained LSTM choosing the desired parameters
- **'Save the Sub-Classifiers (separately)'**: the user can save the new trained LSTM in the output directory
- **'Evaluate the Sub-Classifiers (separately)'**: the user can apply the test dataset to check the performance of the classifier

7.0.4 Main program - Connect Levels - Test

This is where we connect the 2 classification levels. We suppose we have the classifiers we need (either by creating new ones or by loading pre-trained ones), and we want to evaluate the performance of the whole model.

- **'Test First layer'**: we make the predictions on the 1st level
- **'Test Second layer'**: we make the predictions on the 2nd level, using each time the sub-classifier specified by the level-1 prediction
- **'Evaluating:'**: we get the legit answers for both levels and evaluate the performance of the model. We first see the performance of the 1st level alone, and then on both levels.

8 Final Model

8.0.1 Access to the Model

Since the model is too large to fit as a web-mail attachment we uploaded it on **Google Drive**. Here are the access links, one for the level-1 classifier and one for the level-2 classifiers:

- level-1
- level-2

8.0.2 Using the Model

Supposing the user has downloaded the trained model, if he wishes to **use the whole model** on another testing set, he has to follow these steps: load the help functions, execute the cells under the label **'Main Program - Initialization:'** , go to the label **'Main Program - Main Classifier - Load the Main Classifier'** (and adjust the filepath of the model) to load the main classifier, go to the label **'Main Program -Sub-Classifiers - Load the Sub-Classifiers (at once)'** (and adjust the filepath of the sub-models) to load the sub-classifiers, and finally execute all the cells under the label **Connect Levels - Test** (and adjust the filepath of the dataset).

9 Results

The training procedure was done on platforms that offer **GPU accelerators** (Google Colab/Kaggle), so the time statistics we present are based on this and the time performance would be significantly worse without the use use the accelerators.

9.0.1 Training Time/Validation Accuracy

The usage of **inner** dropout in LSTMs proved to put a heavy burden on the system, so we decided to use it for the main classifier which is unique, but we used just regular dropout for the sub-classifiers because they are too many.

For the main classifier we use **25 batches** and a minimum of **25 epochs** and an early stopping in case the validation accuracy does not get better after 7 consecutive epochs. The training time lasts **32.5 minutes**, and the validation accuracy reaches about **85.09%**:

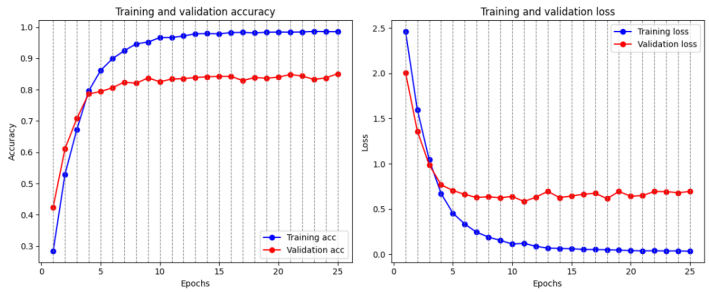

Figure 7: Training History

For the sub-classifiers, as we mentioned, se do not use inner dropout in the LSTM layers in order to save some time. The size of batches was decided experimentally, and since the training data is much fewer for a sub-classifier we used small numbers **(5-10)**. If we use the option to train them all at once with a loop, the total training time is **39 minutes** and the **mean** validation accuracy reaches about **85.4%**

9.0.2 Inference Time/Testing Accuracy

We now used the testing dataset to see the generalization ability of the model. The inference time is **2.3 minutes**, and we get the following testing accuracies:

- level 1 accuracy: **82%**
- both levels accuracy: **66%**

For the 1st level classifier we get the following metrcs:

	precision	recall	f1-score	support
0	0.84	0.87	0.85	30
1	0.89	0.91	0.90	80
2	0.81	0.70	0.75	50
3	0.66	0.62	0.64	50
4	0.89	0.78	0.83	40
5	0.75	0.85	0.80	61
6	0.73	0.90	0.81	60
7	0.91	0.84	0.87	70
8	0.79	0.88	0.83	60
9	0.89	0.91	0.90	70
10	0.87	0.87	0.87	30
11	0.73	0.78	0.75	90
12	0.78	0.76	0.77	80
13	0.78	0.70	0.74	80
14	0.84	0.75	0.79	110
15	0.90	0.93	0.92	91
16	0.92	0.88	0.90	40
accuracy			0.82	1092
macro avg	0.82	0.82	0.82	1092
weighted avg	0.82	0.82	0.82	1092

Average accuracy: 0.82

Figure 8: Performance Metrics

Confusion Matrix:

[[26	0	0	0	0	1	0	0	0	0	0	1	1	0	1	0	0]
[0	73	1	2	0	1	0	0	0	0	0	2	1	0	0	0	0]
[0	0	35	3	0	2	1	0	0	1	0	2	1	3	2	0	0]
[0	2	0	31	1	2	6	0	0	1	0	1	0	2	0	1	3]
[0	0	0	1	31	0	1	1	0	1	0	2	0	1	2	0	0]
[0	1	0	0	0	52	0	0	0	1	0	0	3	1	2	1	0]
[0	0	0	0	0	0	54	0	2	0	0	1	1	2	0	0	0]
[0	0	0	0	0	1	2	59	0	0	1	2	1	3	1	0	0]
[0	0	0	1	0	0	1	0	53	0	0	1	0	2	2	0	0]
[0	0	1	0	0	1	0	0	0	64	0	3	0	1	0	0	0]
[0	0	0	0	0	0	0	0	1	0	26	0	0	0	1	2	0]
[2	4	1	0	1	3	1	0	1	1	0	70	3	0	2	1	0]
[0	1	3	1	0	4	0	0	3	1	0	4	61	1	1	0	0]
[1	0	0	1	2	1	7	1	3	0	0	2	1	56	2	3	0]
[0	1	2	2	0	1	0	3	3	2	2	5	5	0	83	1	0]
[2	0	0	1	0	0	0	1	1	0	1	0	0	0	0	85	0]
[0	0	0	4	0	0	1	0	0	0	0	0	0	0	0	0	35]]

Figure 9: Confusion Matrix

As we can see the metrics are similar across the classes so we assume the the use of class weights did indeed contribute to fixing the data imbalance.

10 Future Work/Notes

Future extensions could include: the use of different pre-trained embedding per sub-classifier, specified in the context of this particular main class.

While training a new model there user can choose if he wants to use a pre-trained embedding with the flag "pre_trained_embedding" used when the function "build_trained_lstm_model". In order to use the pre-trained embedding the

user must have **downloaded Word2Vec**. Here is a link to download it: link

In relation to creating a separate module that preprocesses the input data: unfortunately we could not implement this because we encountered problems while trying to save and load the tokenizer in and out of a file.